

UGREEN SecurityWatch

Installationshandbuch für UGOS

Deutsches Handbuch / English Manual - Version V3.0 - Stand 18.05.2026

Community-Lösung, kein offizielles UGREEN-Produkt. Verwendung auf eigene Verantwortung.

Hinweis / Note: Die englische Version beginnt nach dem deutschen Teil. / The English version starts after the German section.

Changelog

Version	Datum / Date	Highlights
V3.0	2026-05-18	DE: Major-Release mit automatischem Rebuild lokal selbst gebauter Docker-Images, Nachscan nach dem Rebuild, klaren Ergebniszuständen für behobene und weiterhin vorhandene Funde, 7-Tage-Cooldown für unveränderte Funde, Bereinigung leerer Mülldateien im SecurityWatch-Ordner, korrigierter UTF-8/Umlaut-Ausgabe in Logs und Reports sowie verbesserter Report-Darstellung für Cron-Betrieb. Der automatische Rebuild nutzt optional --pull und --no-cache und vermeidet dadurch unnötige nächtliche Wiederholungsbuidls bei Upstream-Funden. EN: Major release with automatic rebuilds for locally built Docker images, post-rebuild rescan, clear result states for fixed and still existing findings, 7-day cooldown for unchanged findings, cleanup of empty stray files in the SecurityWatch folder, corrected UTF-8/umlaut output in logs and reports, and improved report presentation for cron usage. The automatic rebuild can use --pull and --no-cache and avoids unnecessary nightly rebuild loops for upstream findings.
V2.4.3	2026-04-26	DE: Trivy-Image Auto-Pull, sichere Docker-Bereinigung vor dem Scan, optionales Build-Cache-Cleanup, Zusatzreport für lokal selbst gebaute Images, Report-Aufräumen nach Aufbewahrungsregeln und neue Startoptionen wie --dry-run, --no-cleanup, --no-mail und --no-report-cleanup. Die Detailkonfiguration ist in .env.example dokumentiert. EN: Trivy image auto-pull, safe Docker cleanup before the scan, optional build-cache cleanup, additional report for locally built images, report cleanup based on retention rules and new startup options such as --dry-run, --no-cleanup, --no-mail and --no-report-cleanup. Detailed configuration is documented in .env.example.
V1.0	2026-04-04	Erster Release / First release

0. Wichtiger Hinweis zu Release-Struktur und GitHub-Installation

Dieses Handbuch beschreibt die Installation des SecurityWatch-Release-Pakets auf einem UGREEN NAS mit UGOS. Das Paket enthält die Vorlage `.env.example`. Auf der NAS wird daraus anschließend per SSH die aktive Datei `.env` erstellt.

- Die Installation erfolgt in einem festen Arbeitsordner, zum Beispiel `/volume2/docker/securitywatch`.
- Die Ordner `reports`, `trivy-cache` und `state` werden beim ersten Lauf automatisch erzeugt.
- Das Skript nutzt den Docker-Client der NAS, um den offiziellen Trivy-Container für Host- und Image-Scans auszuführen.
- Die Sprache der Ausgaben, Logs und Mails wird in der `.env` über `OUTPUT_LANG=de` oder `OUTPUT_LANG=en` gesteuert.

```
Release-Root (Beispiel)
securitywatch/
├── .env.example
├── securitywatch_scan.sh
├── securitywatch_report.py
└── securitywatch_mail.py
```

0.1 Schnellstart (DE)

- SSH in UGOS aktivieren.
- Mit PuTTY als Administrator-Benutzer auf die NAS verbinden.
- Mit `sudo -i` Root-Rechte anfordern.
- Ordner `/volume2/docker/securitywatch` anlegen und das GitHub-Release dorthin kopieren.
- `.env.example` per `cp .env.example .env` in die aktive `.env` umwandeln.
- SMTP- und Sprachwerte in `.env` anpassen, besonders `OUTPUT_LANG`.
- Einen manuellen Testlauf mit `./securitywatch_scan.sh` ausführen.
- Danach den automatischen Lauf per `crontab -e` einrichten.

1. Zweck und Überblick

UGREEN SecurityWatch ist ein leichtgewichtiges Skriptpaket für UGOS. Es scannt das Host-Dateisystem sowie die Docker-Images laufender Container mit Trivy, erzeugt daraus eine Zusammenfassung und versendet anschließend eine Text- und HTML-Mail.

- Host-Scan per Trivy rootfs gegen `/host`
- Image-Scan der aktuell laufenden Container
- Zusammenfassung in `summary.json`, `mail.txt` und `mail.html`
- Sprachumschaltung über `OUTPUT_LANG=de` oder `OUTPUT_LANG=en`
- Berichte pro Lauf unter `/volume2/docker/securitywatch/reports/<Datum_Uhrzeit>/`

2. Voraussetzungen

- UGREEN NAS mit UGOS
- Ein Administratorkonto mit `sudo`-Rechten
- Aktivierter SSH-Zugang
- PuTTY oder ein anderer SSH-Client auf dem Windows-PC

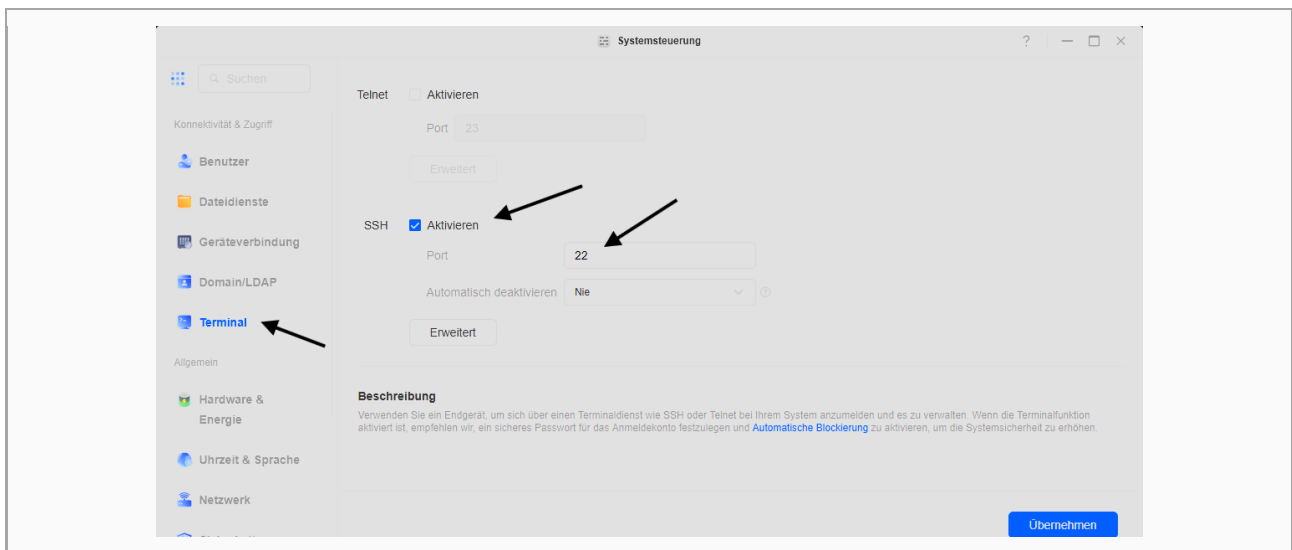
- Funktionsfähiger Docker-Befehl auf der NAS
- Internet-Zugriff für das Trivy-Image und die Trivy-Datenbank
- SMTP-Zugangsdaten für den Mailversand

Praxis-Hinweis: Für den ersten Lauf sollte die NAS einige Minuten Zeit haben. Trivy lädt beim ersten Start Daten nach, deshalb dauert der erste Testlauf oft spürbar länger als spätere Läufe.

3. SSH in UGOS aktivieren

Melde dich in der UGOS-Oberfläche an und öffne Systemsteuerung > Terminal. Aktiviere dort den SSH-Service, prüfe den Port und übernimm die Einstellung. Standardmäßig ist Port 22 üblich. Für mehr Sicherheit empfiehlt UGREEN, den Port bewusst zu wählen und SSH nicht direkt aus dem Internet erreichbar zu machen.

- UGOS öffnen
- Systemsteuerung > Terminal aufrufen
- SSH-Service aktivieren
- Port prüfen oder anpassen
- Mit Anwenden speichern

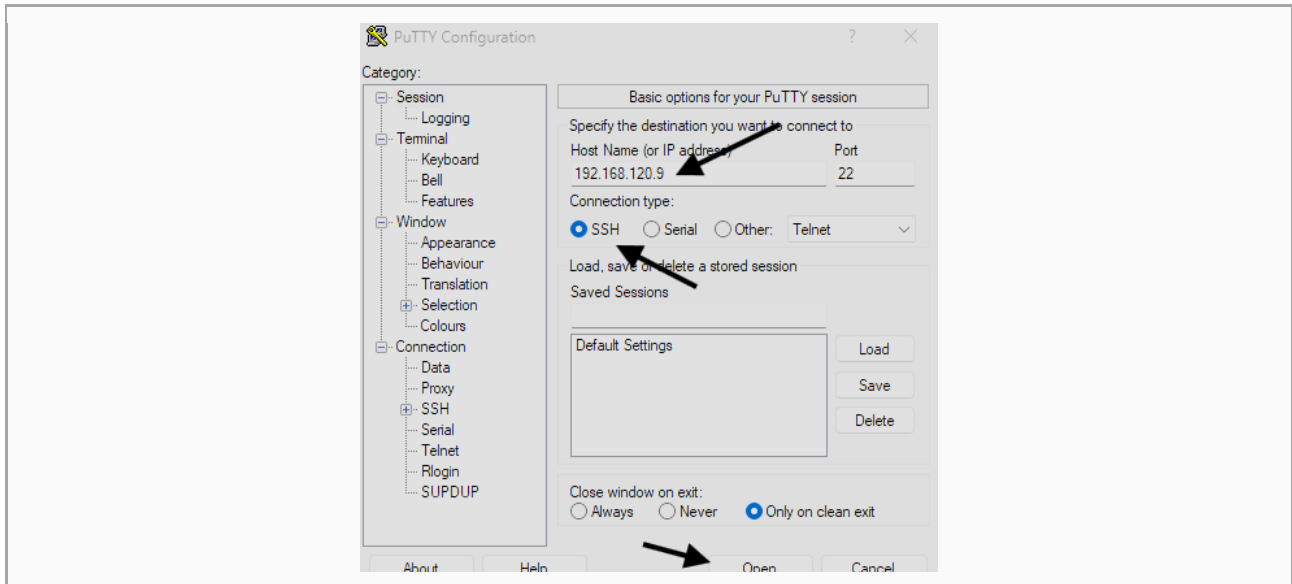


4. Verbindung mit PuTTY herstellen

Starte PuTTY auf deinem Windows-PC. Trage unter Host Name die IP-Adresse deiner NAS ein, unter Port den in UGOS konfigurierten SSH-Port und lasse als Verbindungstyp SSH ausgewählt. Danach klickst du auf Open. Melde dich mit deinem UGOS-Administratorbenutzer an und gib das Passwort ein.

- Host Name: IP-Adresse der NAS
- Port: derselbe Port wie in UGOS unter Terminal
- Connection type: SSH
- Login: dein Administrator-Benutzername
- Passwort: dein UGOS-Passwort

Beispiel für eine SSH-Verbindung
Host Name: 192.168.120.9
Port: 22
Connection type: SSH



5. Nach dem Login Root-Rechte mit sudo -i holen

Nach der normalen SSH-Anmeldung arbeitest du zunächst als dein Benutzerkonto. Für die Installation und den Cronjob ist es in der Praxis am einfachsten, anschließend Root-Rechte mit sudo -i anzufordern. Danach ändert sich die Shell auf Root-Ebene.

```
sudo -i

# optional zur Kontrolle
whoami
```

Wichtig: Root-Rechte geben vollständigen Zugriff auf das System. Verwende sudo -i deshalb bewusst und nur für administrative Schritte.

6. Projektordner auf der NAS anlegen und Release kopieren

Lege den Zielordner für SecurityWatch an. Wenn du das GitHub-Release zuerst auf den PC heruntergeladen hast, kannst du die Dateien anschließend per Netzwerkfreigabe, File Station oder SCP in diesen Ordner kopieren.

```
sudo -i
mkdir -p /volume2/docker/securitywatch
cd /volume2/docker/securitywatch
ls -la
```

Empfohlene Zielstruktur nach dem Kopieren:

```
/volume2/docker/securitywatch/  
├── .env.example  
├── securitywatch_scan.sh  
├── securitywatch_report.py  
└── securitywatch_mail.py
```

7. .env vorbereiten und anpassen

Im Paket ist die Vorlage `.env.example` enthalten. Erstelle daraus die aktive Datei `.env` und bearbeite danach die wichtigsten Variablen.

```
cd /volume2/docker/securitywatch  
cp .env.example .env  
nano .env
```

Variable	Beispiel	Bedeutung
OUTPUT_LANG	de	Sprache für Logs, Text-Mail und HTML-Mail. Erlaubte Werte: de oder en.
BASE_DIR	/volume2/docker/ /securitywatch	Basispfad des Projekts.
REPORT_DIR	/volume2/docker/ /securitywatch/re ports	Speicherort für Laufberichte.
CACHE_DIR	/volume2/docker/ /securitywatch/tr ivy-cache	Trivy-Cache. Wird automatisch aufgebaut.
STATE_DIR	/volume2/docker/ /securitywatch/st ate	Zustandsdaten des Pakets.
SMTP_SERVER	mail.example.co m	SMTP-Server für den Versand.
SMTP_FROM / SMTP_TO	nas@example.co m	Absender- und Empfängeradresse.
SMTP_PASS_FILE	/volume2/docker/ /securitywatch/.s mtp_pass	Optionaler Pfad zu einer Passwortdatei statt Klartext im ENV.
MAIL_ONLY_ON_F INDINGS	1	Bei 1 wird nur bei Treffern versendet.
MAIL_SEND_OK	0	Bei 1 darf auch eine OK-Mail ohne Treffer versendet werden.

Empfehlung: Für einen reinen Funktionstest des Mailversands kannst du MAIL_ONLY_ON_FINDINGS=0 und MAIL_SEND_OK=1 setzen. Danach erhältst du auch dann eine Mail, wenn im Testlauf keine High- oder Critical-Funde vorliegen.

8. Erster manueller Testlauf

Vor dem Cronjob sollte immer ein manueller Testlauf erfolgen. So siehst du direkt, ob Docker, Trivy, Pfade und SMTP korrekt funktionieren.

```
cd /volume2/docker/securitywatch
chmod +x securitywatch_scan.sh securitywatch_report.py securitywatch_mail.py
./securitywatch_scan.sh
```

- Beim ersten Lauf kann das Herunterladen der Trivy-Datenbank zusätzliche Zeit benötigen.
- Die Berichte landen im neu angelegten Unterordner unter reports.
- Je nach .env-Einstellung wird eine Mail nur bei Treffern oder auch als OK-Meldung versendet.

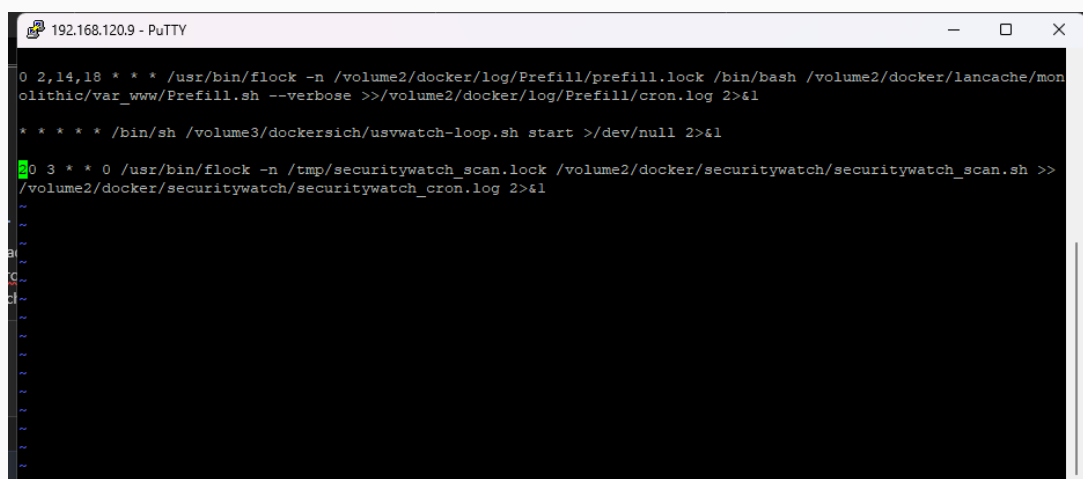
9. Cronjob mit crontab -e einrichten

Wenn der manuelle Test erfolgreich war, richtest du den automatischen Lauf per Cron ein. Öffne dazu den Crontab-Editor für Root und ergänze eine Zeile mit vollständigen Pfaden.

```
sudo -i
crontab -e
```

Beispiel: täglicher Lauf um 06:30 Uhr

```
30 6 * * * cd /volume2/docker/securitywatch &&
/volume2/docker/securitywatch/securitywatch_scan.sh >>
/volume2/docker/securitywatch/cron.log 2>&1
```

A screenshot of a terminal window titled '192.168.120.9 - PuTTY'. The terminal shows a series of commands being entered and executed. The first command is '0 2,14,18 * * * /usr/bin/flock -n /volume2/docker/log/Prefill/prefill.lock /bin/bash /volume2/docker/lancache/monolith/var_www/Prefill.sh --verbose >>/volume2/docker/log/Prefill/cron.log 2>&1'. The second command is '* * * * /bin/sh /volume3/dockersich/uswatch-loop.sh start >/dev/null 2>&1'. The third command is '0 3 * * 0 /usr/bin/flock -n /tmp/securitywatch_scan.lock /volume2/docker/securitywatch/securitywatch_scan.sh >>/volume2/docker/securitywatch/securitywatch_cron.log 2>&1'. The terminal output shows the execution of these commands, with some lines wrapped. The terminal window has standard window controls (minimize, maximize, close) in the top right corner.

9.1 Bedeutung der fünf Zeitfelder in Cron

Position	Feld	Erlaubte Werte	Bedeutung
1	Minute	0-59	Zu welcher Minute der Befehl startet.
2	Stunde	0-23	Zu welcher Stunde der Befehl startet.
3	Tag des Monats	1-31	Welcher Kalendertag im Monat gelten soll.
4	Monat	1-12 oder Namen	In welchem Monat der Job läuft.
5	Wochentag	0-7 oder Namen	Wochentag. 0 oder 7 steht für Sonntag.

Die wichtigsten Platzhalter und Operatoren:

- * bedeutet immer bzw. jeder mögliche Wert.
- 1,3,5 bedeutet eine Liste einzelner Werte.
- 1-5 bedeutet einen Bereich, hier 1 bis 5.
- */2 bedeutet Schrittweite, hier jeder zweite Wert.
- Wenn sowohl Tag des Monats als auch Wochentag eingeschränkt sind, wird der Job ausgeführt, wenn eines der beiden Felder passt.

9.2 Lesebeispiele für typische Cron-Einträge

Eintrag	Bedeutung
30 6 * * *	Jeden Tag um 06:30 Uhr
0 3 * * 1	Jeden Montag um 03:00 Uhr
15 2 1 * *	Am ersten Tag jedes Monats um 02:15 Uhr
0 */6 * * *	Alle sechs Stunden zur vollen Stunde

Best Practice: Verwende im Cronjob immer vollständige Pfade. Der Cron-Dienst läuft mit einer deutlich kleineren Umgebung als eine interaktive Shell.

10. Reports, Logs und automatisch erzeugte Ordner prüfen

Nach dem ersten Lauf findest du unterhalb von BASE_DIR mehrere automatisch angelegte Ordner und Dateien. Diese müssen nicht im GitHub-Release enthalten sein, entstehen aber bei der Nutzung automatisch.

Ordner / Datei	Zweck
reports	Enthält für jeden Lauf einen Zeitstempel-Ordner mit JSON-, Log- und Mail-Dateien.
trivy-cache	Zwischenspeicher für Trivy-Datenbank und Scan-Metadaten.

state	Zustandsdaten des Pakets.
cron.log	Optionales Sammellog des Cronjobs, wenn du die Umleitung aus dem Beispiel übernimmst.

```
cd /volume2/docker/securitywatch
ls -lah
find reports -maxdepth 2 -type f | sort
```

11. Update des Pakets

Für ein Update ersetzt du in der Regel nur die Skriptdateien und gegebenenfalls .env.example. Die aktive .env mit deinen Zugangsdaten bleibt auf der NAS erhalten. Nach einem Update empfiehlt sich erneut ein manueller Testlauf.

- Neue Release-Dateien in den bestehenden Ordner kopieren
- Bestehende .env nicht überschreiben
- Dateirechte mit chmod +x bei Bedarf erneut setzen
- Skript einmal manuell testen
- Danach wieder den normalen Cronjob laufen lassen

12. Troubleshooting

Problem	Mögliche Ursache	Praktische Lösung
PuTTY verbindet nicht	SSH in UGOS nicht aktiv oder falscher Port	SSH-Einstellung und Port in UGOS prüfen.
sudo -i funktioniert nicht	Benutzer ist kein Administrator	Mit einem administrativen UGOS-Benutzer anmelden.
Keine Mail	SMTP-Werte unvollständig oder Versand nur bei Treffern	SMTP prüfen, MAIL_ONLY_ON_FINDINGS und MAIL_SEND_OK kontrollieren.
Cronjob läuft nicht	Falscher Pfad oder Tippfehler im Crontab	crontab -e erneut öffnen und Eintrag exakt prüfen.
Erster Lauf dauert lange	Trivy lädt Datenbank oder scannt viele Images	Abwarten. Spätere Läufe sind meist schneller.

UGREEN SecurityWatch - Installation Manual for UGOS

English version - Version V3.0 - Updated 2026-05-18

Community solution, not an official UGREEN product. Use at your own risk.

0. Important note about release structure and GitHub installation

The package contains a template file named `.env.example`. Create the active `.env` from it and then adjust the most important variables. On the NAS, that file is then turned into the active `.env` via SSH.

- Installation is done in a fixed working directory, for example `/volume2/docker/securitywatch`.
- The folders `reports`, `trivy-cache` and `state` are created automatically during the first run.
- The script uses the NAS Docker client to run the official Trivy container for host and image scans.
- The language of logs, outputs and emails is controlled in `.env` via `OUTPUT_LANG=de` or `OUTPUT_LANG=en`.

```
Release root (example)
securitywatch/
├── .env.example
├── securitywatch_scan.sh
├── securitywatch_report.py
└── securitywatch_mail.py
```

0.1 Quick start (EN)

- Enable SSH in UGOS.
- Connect to the NAS with PuTTY using an administrator account.
- Request root privileges with `sudo -i`.
- Create `/volume2/docker/securitywatch` and copy the GitHub release there.
- Turn `.env.example` into the active `.env` with `cp .env.example .env`.
- Adjust SMTP settings and `OUTPUT_LANG` in `.env`.
- Run a first manual test with `./securitywatch_scan.sh`.
- Then create the scheduled job with `crontab -e`.

1. Purpose and overview

UGREEN SecurityWatch is a lightweight script package for UGOS. It scans the host filesystem and the Docker images of running containers with Trivy, builds a summary and then sends a plain text and HTML email.

- Host scan via Trivy rootfs against /host
- Image scan of currently running containers
- Summary in summary.json, mail.txt and mail.html
- Language switch via OUTPUT_LANG=de or OUTPUT_LANG=en
- One report folder per run under /volume2/docker/securitywatch/reports/<date_time>/

2. Requirements

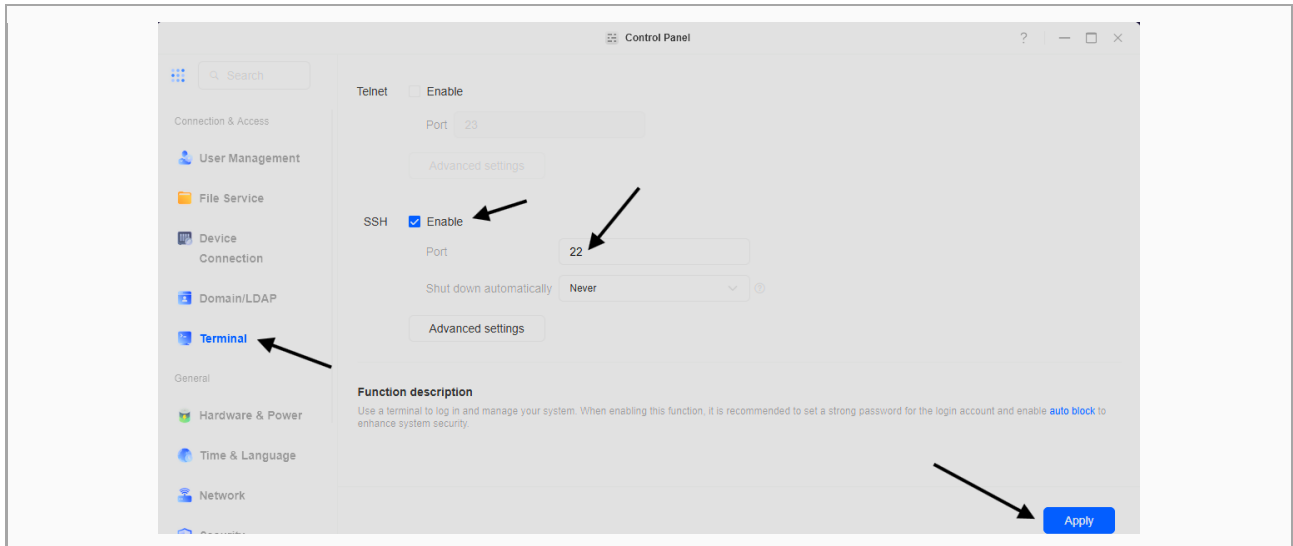
- UGREEN NAS running UGOS
- An administrator account with sudo rights
- SSH access enabled
- PuTTY or another SSH client on the Windows PC
- A working Docker CLI on the NAS
- Internet access for the Trivy image and Trivy database
- SMTP credentials for email delivery

Practical note: Allow some extra time for the first run. Trivy may need to download data on first start, so the initial test often takes noticeably longer than later runs.

3. Enable SSH in UGOS

Log in to the UGOS interface and open Control Panel > Terminal. Enable the SSH service there, review the port and apply the setting. Port 22 is common by default. For better security, UGREEN recommends choosing the port deliberately and not exposing SSH directly to the internet.

- Open UGOS
- Go to Control Panel > Terminal
- Enable SSH Service
- Review or change the port
- Save with Apply



4. Connect with PuTTY

Start PuTTY on your Windows PC. Enter the NAS IP address in Host Name, the SSH port configured in UGOS in Port, and keep SSH selected as the connection type. Then click Open. Log in with your UGOS administrator user and password.

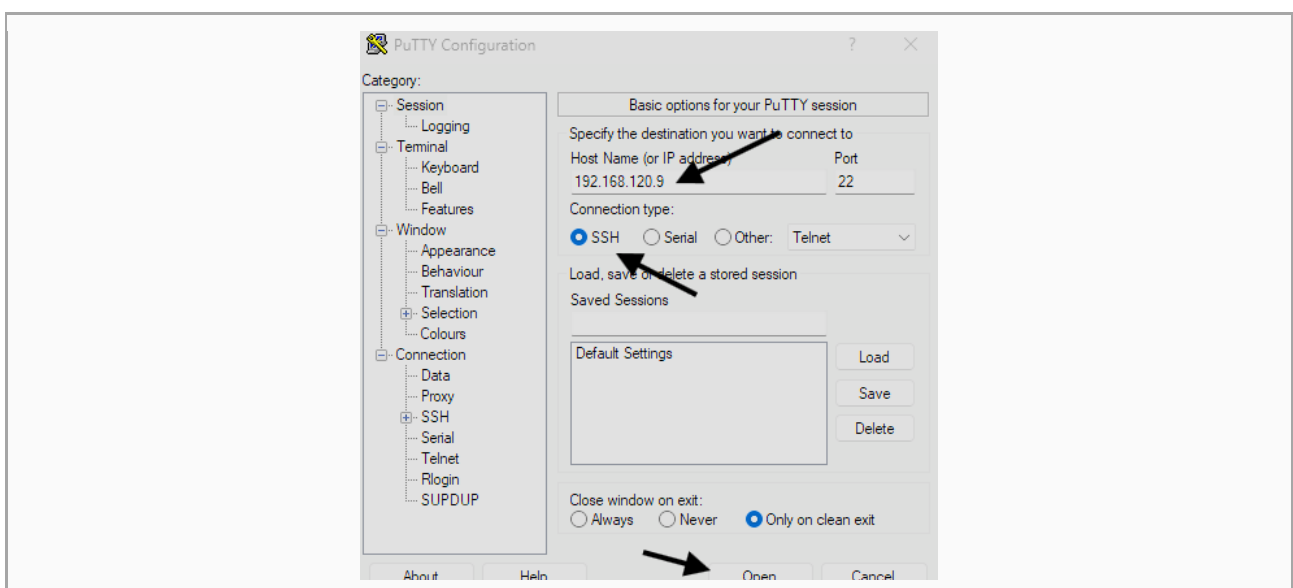
- Host Name: IP address of the NAS
- Port: same port configured in UGOS under Terminal
- Connection type: SSH
- Login: your administrator username
- Password: your UGOS password

Example SSH connection values

Host Name: 192.168.120.9

Port: 22

Connection type: SSH



5. Request root privileges with sudo -i

After the normal SSH login you initially work as your regular user. For installation steps and the cron job it is usually easiest to request root privileges afterwards with `sudo -i`. The shell then switches to root level.

```
sudo -i

# optional verification
whoami
```

Important: Root privileges provide full control over the system. Use `sudo -i` carefully and only for administrative steps.

6. Create the project directory and copy the release

Create the target directory for SecurityWatch. If you downloaded the GitHub release to the PC first, you can then copy the files by network share, File Station or SCP into that folder.

```
sudo -i
mkdir -p /volume2/docker/securitywatch
cd /volume2/docker/securitywatch
ls -la
```

Recommended target structure after copying:

```
/volume2/docker/securitywatch/
├── .env.example
├── securitywatch_scan.sh
├── securitywatch_report.py
└── securitywatch_mail.py
```

7. Prepare and edit .env

The GitHub release should contain a template file named `.env.example`. Create the active `.env` from it and then edit the most important variables.

```
cd /volume2/docker/securitywatch
cp .env.example .env
nano .env
```

Variable	Example	Meaning
OUTPUT_LANG	en	Language for logs, plain text email and HTML email. Allowed values: de or en.

BASE_DIR	/volume2/docker /securitywatch	Base path of the project.
REPORT_DIR	/volume2/docker /securitywatch/re ports	Storage location for run reports.
CACHE_DIR	/volume2/docker /securitywatch/tr ivy-cache	Trivy cache. Built automatically.
STATE_DIR	/volume2/docker /securitywatch/st ate	State data of the package.
SMTP_SERVER	mail.example.co m	SMTP server used for sending.
SMTP_FROM / SMTP_TO	nas@example.co m	Sender and recipient address.
SMTP_PASS_FILE	/volume2/docker /securitywatch/.s mtp_pass	Optional path to a password file instead of storing the password directly in the env.
MAIL_ONLY_ON_F INDINGS	1	When set to 1, mail is sent only when findings exist.
MAIL_SEND_OK	0	When set to 1, an OK mail without findings may also be sent.

Recommendation: For a pure mail function test, you can temporarily set MAIL_ONLY_ON_FINDINGS=0 and MAIL_SEND_OK=1. That way you receive a mail even if the test run reports no High or Critical findings.

8. First manual test run

Always do a manual test before enabling the cron job. This confirms that Docker, Trivy, paths and SMTP work correctly.

```
cd /volume2/docker/securitywatch
chmod +x securitywatch_scan.sh securitywatch_report.py securitywatch_mail.py
./securitywatch_scan.sh
```

- The first run may take extra time because the Trivy database may need to be downloaded.
- Reports are written into the newly created reports subfolder.
- Depending on the .env settings, email is sent only on findings or also as an OK notification.

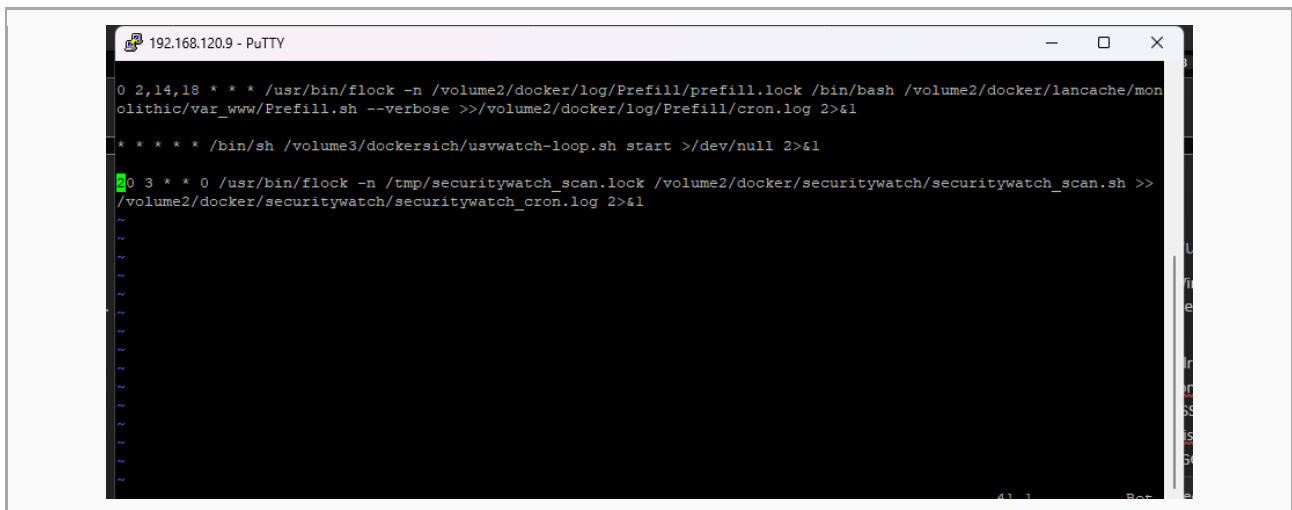
9. Create the cron job with crontab -e

When the manual test succeeds, configure the automatic run with cron. Open the root crontab editor and add a line with full paths.

```
sudo -i  
crontab -e
```

Example: daily run at 06:30

```
30 6 * * * cd /volume2/docker/securitywatch &&  
/volume2/docker/securitywatch/securitywatch_scan.sh >>  
/volume2/docker/securitywatch/cron.log 2>&1
```



9.1 Meaning of the five cron time fields

Position	Field	Allowed values	Meaning
1	Minute	0-59	At which minute the command starts.
2	Hour	0-23	At which hour the command starts.
3	Day of month	1-31	Which calendar day in the month applies.
4	Month	1-12 or names	In which month the job runs.
5	Day of week	0-7 or names	Weekday. 0 or 7 means Sunday.

The most important placeholders and operators:

- * means every possible value.
- 1,3,5 means a list of individual values.
- 1-5 means a range, here 1 through 5.

- */2 means a step value, here every second value.
- If both day-of-month and day-of-week are restricted, the job runs when either of the two fields matches.

9.2 Reading examples for typical cron entries

Entry	Meaning
30 6 * * *	Every day at 06:30
0 3 * * 1	Every Monday at 03:00
15 2 1 * *	On the first day of each month at 02:15
0 */6 * * *	Every six hours on the hour

Best practice: Always use full paths in cron jobs. The cron service runs with a much smaller environment than an interactive shell.

10. Check reports, logs and automatically created folders

After the first run, several folders and files will exist below BASE_DIR automatically. They do not need to be part of the GitHub release, but they are created during normal use.

Folder / file	Purpose
reports	Contains one timestamped folder per run with JSON, log and mail files.
trivy-cache	Cache for the Trivy database and scan metadata.
state	State data used by the package.
cron.log	Optional combined log of the cron job if you keep the redirection from the example.

```
cd /volume2/docker/securitywatch
ls -lah
find reports -maxdepth 2 -type f | sort
```

11. Updating the package

For updates you usually replace only the script files and, if needed, .env.example. The active .env containing your credentials stays on the NAS. After an update, another manual test run is recommended.

- Copy the new release files into the existing folder
- Do not overwrite the existing .env
- Reapply chmod +x if necessary
- Run the script once manually
- Then let the normal cron job continue

12. Troubleshooting

Problem	Possible cause	Practical fix
PuTTY cannot connect	SSH disabled in UGOS or wrong port	Verify SSH and the configured port in UGOS.
sudo -i fails	User is not an administrator	Log in with an administrative UGOS user.
No email	Incomplete SMTP values or mail only on findings	Check SMTP and review MAIL_ONLY_ON_FINDINGS and MAIL_SEND_OK.
Cron job does not run	Wrong path or typo in crontab	Open crontab -e again and verify the line exactly.
First run is slow	Trivy downloads its database or many images are scanned	Wait for completion. Later runs are often faster.